Your economic data project

A beginner's guide to the code, files and data pipeline
Capstone project | October 25, 2026

This project is a small automated research assistant for economic data. It downloads numbers from public sources, organizes them, calculates economic variables, and produces spreadsheets plus reports about data quality.

The main dataset covers 12 emerging-market countries from January 2000 through December 2019. An optional extension adds U.S. economic observations for the same months.

The overall journey

```
Public data APIs
       |
       v
data/raw/         Saved responses and request details
       |
       v
data/normalized/  Standardized individual series
       |
       v
data/processed/   Final research tables
       +
reports/          Coverage, missing values and source notes
```

A **pipeline** is simply a sequence of steps that turns source data into useful output. Keeping each stage separate makes it easier to inspect the data and repeat the work.

What the final table looks like

A **panel** contains repeated observations for multiple subjects. Here, each row represents one country in one month. Columns contain that month's exchange rate, CPI, production, credit and calculated variables.

```
country    date       nominal_fx    domestic_cpi    ...
Brazil     2000-01    ...           ...
Brazil     2000-02    ...           ...
Mexico     2000-01    ...           ...
```

The main panel has 12 countries x 240 months = **2,880 rows**. Rows remain in the table even when some values are missing. A full set of rows does not mean all observations are available or all definitions have been validated.

Suggested reading route

Start with download.py, then providers.py, then data_finder.py, then audit.py. Together, these show the central journey from requesting data to calculating the final variables.

1. The folder structure

```
code/
  README.md                 Project instructions and research notes
  requirements.txt          Required Python libraries
  requirements-tested.txt   Exact versions previously tested
  setup_fred.py             Configure the FRED API key
  discover.py               Explore emerging-market sources
  download.py               Download and standardize selected data
  data_finder.py            Build the emerging-market panel
  discover_us.py            Explore U.S. sources
  us_data_finder.py          Start the U.S. build

  eme_pipeline/             Reusable Python tools
    __init__.py
    config.py
    settings.py
    http.py
    providers.py
    audit.py
    reporting.py
    fred.py
    us.py

  data/
    raw/                    Saved provider responses
    normalized/             Individual standardized CSV tables
    processed/              Final assembled datasets

  reports/                  Quality checks and research explanations
  tests/                    Automated checks of code behavior
```

Scripts versus reusable tools

The Python files at the top level are mostly **scripts you run**. They coordinate a workflow. Files inside eme_pipeline/ provide **reusable tools** those scripts call. This avoids rewriting the same download or calculation logic in several places.

Supporting and automatically generated files

README.md is the project manual. **requirements.txt** lists pandas, NumPy, requests and pytest. **requirements-tested.txt** records exact dependency versions used in a checked environment.

.env stores your local FRED key; **.env.example** shows the expected setting. **.gitignore** tells Git which files to exclude, including .env. **.git/** holds version-control history.

.venv/ is the project's isolated Python environment. **.pytest_cache/** and **__pycache__/** contain automatically generated helper files. **.DS_Store** is a macOS folder-settings file.

2. A little Python vocabulary

Term	Meaning in this project
Variable	A name holding a value, such as a country code or a table.
Function	A named, reusable task, such as downloading one series.
Class	A blueprint for an object that groups related tools and settings.
DataFrame	A pandas table with rows and columns, similar to a worksheet.
Series	A pandas object resembling one column of values.
API	A service your code asks for data over the internet.
CSV	A plain-text table that Excel can open.
JSON	A structured format used for API responses and saved details.
Metadata	Information describing data: source, units, frequency and dates.
Cache	Saved information reused instead of downloading it again.

Loading libraries and project tools

```
import pandas as pd
from eme_pipeline.http import Client
```

The first line loads the pandas library and gives it the short name pd. The second loads the Client tool from your own project's eme_pipeline/http.py file.

Defining and starting a function

```
def main():
    # Instructions go here
    ...

if __name__ == '__main__':
    main()
```

def defines a function; it does not immediately run its instructions. The final two lines mean: "If someone runs this file directly, start its main() function." Indentation groups instructions together.

Loops, dictionaries and error handling

A **for loop** repeats a task, such as downloading a series for each country. A **dictionary** associates keys with values, such as BRA with Brazil. A **try/except** block attempts work and handles a failure so the program can record it or continue.

3. The scripts you run

setup_fred.py - configure access to FRED

FRED provides U.S. economic data. Its official API needs an access credential called an API key. This script prompts for the key without displaying it, validates its format, and saves it in .env. The --status option checks whether a key is configured. It does not test whether FRED accepts the key.

discover.py - investigate emerging-market sources

Think of this as looking through a library catalog before choosing books. It requests catalogs, descriptions and sample series from sources such as the World Bank, IMF and BCRP. Options explore different candidates. Responses are saved for inspection; some investigated sources never become final panel inputs.

download.py - collect selected data

This loops through countries and requests exchange rates, CPI, industrial production, private credit and available selected sovereign spreads. Provider adapters convert responses to standard tables, which are written under data/normalized/.

If one series fails, unrelated downloads continue. Failures are recorded in reports/download_status.json. An older normalized file for a failed series is renamed with .stale.csv so the builder will not quietly reuse it. The script ends with a failure exit status if any download or parsing job failed.

data_finder.py - assemble and audit the EME panel

This reads normalized CSVs, aligns their dates, calculates research variables, identifies missing observations, and measures how many months contain all four required variables. It writes data/processed/eme_monthly_panel.csv plus quality reports. Despite its name, its main role is building and auditing the panel.

The default EME build works locally without internet access. Adding --include-us also runs the optional U.S. extension; --us-offline makes that extension use saved responses.

discover_us.py - inspect U.S. sources

This retrieves FRED descriptions, including frequency, units, seasonal adjustment and available dates. It also investigates IMF U.S. credit candidates and saves discovery results. It helps determine which series are suitable before building the U.S. table.

us_data_finder.py - launch the U.S. build

This short script reads the --offline option, creates a Client, and calls build() in eme_pipeline/us.py. The original EME panel must exist first because the extension also creates a combined EME-U.S. panel.

```
build(Client(offline=args.offline))
```

Read this as: "Build the U.S. dataset using this configured download and cache tool."

4. Reusable tools: settings and retrieval

eme_pipeline/__init__.py - identify the package

This contains a brief description and marks the folder as a Python package. A package lets other files import its tools using names such as eme_pipeline.http. It performs no data processing itself.

eme_pipeline/config.py - shared project choices

This defines the 12 country codes and names, the expected raw input variables, and the four required variables used for complete-case checks. Keeping these choices together helps the scripts use consistent definitions.

```
COUNTRIES = {'BRA': 'Brazil', 'MEX': 'Mexico', ...}
```

eme_pipeline/settings.py - read and save credentials

get_fred_api_key() looks for the FRED key in the running process's environment first, then in the project .env file. An environment variable is a setting supplied to a program from outside its source code. save_fred_api_key() validates and safely saves a supplied key.

eme_pipeline/http.py - internet requests and caching

This defines Client, the shared tool for requesting and saving responses. Its get() method builds an identity for a request, checks for a successful cached response, verifies its hash, and returns it if available. A hash is a digital fingerprint used to detect changes to saved bytes.

If no successful response is saved, offline mode raises an error. Online mode makes the actual network request with requests.Session.get(). The client adds time limits, retries for certain temporary failures, a short pause before network requests, and credential redaction. It saves a response body and a JSON request manifest. Its json() method also converts returned JSON into Python objects.

Important: successful cached responses are reused even in a normal online run. Rerunning does not automatically download a newer data vintage.

eme_pipeline/providers.py - translate provider formats

Each provider uses different response formats. These adapters translate them into monthly tables with fields such as date, value, frequency and status, retaining extra attributes where available.

wb_pages() handles World Bank pagination. **gem()** retrieves GEM exchange rates, CPI and production. **imf_credit()** retrieves and checks IMF credit; **imf_cpi()** handles IMF CPI. **bcrp()** retrieves native monthly spreads for comparison. **bcrp_daily_monthly()** averages daily spreads by month, requiring at least 15 observed days.

For example, a provider date labeled 2000M01 is standardized to 2000-01. The adapters also reject unexpected frequencies or identifying metadata instead of silently accepting a different series.

5. Reusable tools: calculations and U.S. data

eme_pipeline/audit.py - validate and calculate

clean() checks monthly dates, rejects duplicate months, converts values to numbers and sorts the series. **coverage()** measures which target months contain usable observations. **complete_cases()** finds rows with every required variable.

transform() calculates the main EME variables:

```
real_fx = nominal_fx * us_cpi / domestic_cpi
log_real_fx = ln(real_fx)
log_ip = ln(industrial_production)
real_credit = nominal_credit / (domestic_cpi / 100)
annual_growth = 100 * (real_credit[t] / real_credit[t-12] - 1)
```

The code aligns observations to a complete monthly calendar before taking lags. Going back 12 rows then truly means 12 months. It retains 1999 internally so January 2000 can have annual growth. Nonpositive inputs to logs and certain derived variables are treated as missing.

usd_spread_bp() is a helper for calculating yield differences under specified conditions. The selected panel spreads come from BCRP EMBIG series.

eme_pipeline/reporting.py - explain results and alternatives

This produces candidate audits, source comparisons and written research findings. For example, it compares monthly averages calculated from daily spreads with native monthly spreads. It uses an offline client: API-looking calls here read saved responses rather than making network requests. Investigated candidates are distinguished from selected panel inputs.

eme_pipeline/fred.py - communicate with FRED

The Fred class uses the shared Client. query() attaches credentials and requests an endpoint; metadata() retrieves a series description; monthly() retrieves observations and handles pagination. It checks monthly frequency and dates, rejects duplicate months, and converts FRED's "." missing-value marker into a missing numeric value. Live access requires a key; offline replay does not.

eme_pipeline/us.py - build and attach U.S. data

build() coordinates retrieval, validation, calculations, merging and output. Inputs include FEDFUNDS (interest rate), PCE and PCEC96 (consumption), INDPRO (production), CPIAUCSL (prices), and LOANS plus other credit candidates. validate_metadata() checks units, frequency and adjustment; transform_us() calculates variables; write_us_report() explains results.

The builder tries IMF credit first. If usable IMF observations are unavailable, it selects FRED LOANS as an explicitly labeled proxy. Main U.S. annual credit growth is 100 times a 12-month log difference; an arithmetic version is also provided for comparison with EME growth.

merge_us() joins by month, repeats U.S. values across countries, and checks that rows and original columns are preserved. FRED CPI becomes us_credit_cpi in the combined table, preserving the existing GEM us_cpi.

6. Data outputs, reports and tests

The three data stages

data/raw/ holds response bodies (.body) and request manifests (.json), organized by provider and country. These preserve evidence of what was retrieved and allow replay. Credential values are redacted.

data/normalized/ holds standardized individual series. For example, data/normalized/BRA/nominal_fx.csv is Brazil's exchange-rate table. Separating these tables from the final panel makes parsing and calculations easier to inspect.

data/processed/ holds three final research tables:

```
eme_monthly_panel.csv       Emerging-market observations
us_monthly_macro.csv        One U.S. row per month
eme_us_monthly_panel.csv     EME observations with U.S. columns
```

The most useful reports

Report	What it tells you
research_findings.md	Written explanation of EME results and limitations.
series_coverage.csv	How many usable observations each series has.
missing_months.csv	Exactly which observations are missing.
source_registry.csv	Sources, series definitions and caveats.
country_eligibility.csv	Months with all required variables; research clearance is separate.
download_status.json	Which download jobs succeeded or failed.
credit_lag_coverage.csv	Availability of current and year-earlier inputs.
observation_flags.csv	Large adjacent changes needing review.

Many reports have corresponding us_ versions. U.S. integration integrity records hashes and row counts to check that the original EME file is preserved. Log files record output from earlier runs.

Automated tests

tests/test_audit.py checks calendars, missing values, calculations and complete cases. **tests/test_providers.py** checks parsing, pagination and daily aggregation. **tests/test_settings.py** checks credential handling. **tests/test_us.py** checks U.S. calculations, FRED handling, merging, caching and redaction.

A test supplies known inputs and checks the expected answer. For example, real credit rising from 100 to 110 over a year should produce 10% arithmetic growth. These files check code behavior; economic comparability still requires source review.

7. Follow one observation from start to finish

Suppose you want Brazil's real-credit growth for January 2000. The following sequence connects the files you have just read about.

Step 1 - request the inputs

download.py asks providers.py for Brazil's nominal private credit and CPI. The adapters construct the relevant IMF and World Bank requests.

Step 2 - retrieve or replay the response

providers.py uses http.py. The Client returns verified cached data if available; otherwise it makes a network request when online. It saves the response and request details under data/raw/.

Step 3 - standardize the observations

The adapters parse provider-specific formats into monthly tables. download.py saves those tables under data/normalized/BRA/.

Step 4 - align the dates and calculate

data_finder.py reads the tables. audit.py aligns them to the monthly calendar, calculates real credit for January 1999 and January 2000, and calculates the percentage change between them.

```
Illustrative values only:

January 1999 real credit = 100
January 2000 real credit = 110

Growth = 100 * (110 / 100 - 1) = 10%
```

Step 5 - save the result and its audit trail

The result becomes a value in data/processed/eme_monthly_panel.csv. Coverage and lag reports record whether the required inputs existed. If either month's credit or CPI is missing or invalid, growth stays missing.

Commands that connect the stages

```
# Rebuild EME data from saved responses
.venv/bin/python download.py --offline
.venv/bin/python data_finder.py

# Add the U.S. extension from saved responses
.venv/bin/python us_data_finder.py --offline

# Run the automated checks
.venv/bin/python -m pytest -q
```

Offline commands require matching saved responses. To make live requests, omit --offline; official FRED access also needs the configured key. Successful cached responses are still reused.

Research boundary: this project collects and audits data. It does not fill missing observations, estimate a VAR, identify policy shocks, or certify that every source definition is fully comparable.